Informe Laboratorio 2

Sección 1

Cristóbal Rivera Guzmán
e-mail: cristobal.rivera2@mail.udp.cl

Septiembre de 2024

Índice

1. Descripción de actividades **3**

2. Desarrollo de actividades según criterio de rúbrica **4**
2.1. Levantamiento de docker para correr DVWA (dvwa) 4
2.1.1. Dependencias 4
2.1.2. Desarrollo de actividad 4
2.2. Redirección de puertos en docker (dvwa) 6
2.3. Obtención de consulta a replicar (burp) 6
2.3.1. Dependencias 6
2.3.2. Desarrollo de actividad 7
2.4. Identificación de campos a modificar (burp) 8
2.5. Obtención de diccionarios para el ataque (burp) 8
2.6. Obtención de al menos 2 pares (burp) 8
2.7. Obtención de código de inspect element (curl) 12
2.8. Utilización de curl por terminal (curl) 14
2.9. Demuestra 4 diferencias (curl) 15
2.10. Instalación y versión a utilizar (hydra) 17
2.11. Explicación de comando a utilizar (hydra) 17
2.12. Obtención de al menos 2 pares (hydra) 18
2.13. Explicación paquete curl (tráfico) 18
2.14. Explicación paquete burp (tráfico) 19
2.15. Explicación paquete hydra (tráfico) 19
2.16. Mención de las diferencias (tráfico) 19
2.17. Detección de SW (tráfico) 20
2.18. Interacción con el formulario (python) 20
2.19. Cabeceras HTTP (python) 20

2.20. Obtención de al menos 2 pares (python) 20
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python) 20
2.22. Demuestra 4 métodos de mitigación (investigación) 21

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA
(Damn Vulnerable Web App - https://github.com/digininja/DVWA (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (https://portswigger.net/burp/communitydownload (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
  - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
  - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
  - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
  - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
  - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

2.1.1. Dependencias

para la ejecución de esta sección del laboratorio se requieren las siguientes dependencias

- **docker**: el uso de docker será necesario para levantar la página web DVWA en un contenedor, la versión de docker usada en este laboratorio es la siguiente

```
pisto@fedora:~/Desktop/cripto/lab2$ sudo docker --version
Docker version 29.3.1, build 1.fc43
pisto@fedora:~/Desktop/cripto/lab2$
```

Figura 1: demostración de la versión de docker utilizada

- **docker-compose**: el uso de docker-compose será necesario para levantar las imagenes del repositorio DVWA, la versión de docker-compose usada en este laboratorio es la siguiente

```
pisto@fedora:~/Desktop/cripto/lab2$ docker-compose --version
Docker Compose version 5.1.1
pisto@fedora:~/Desktop/cripto/lab2$
```

Figura 2: demostración de la versión de docker-compose utilizada

- **git**: será necesario el uso de git para clonar el repositorio de DVWA, la ersión de git usada en el laboratorio es la siguiente

```
pisto@fedora:~/Desktop/cripto/lab2$ sudo git --version
git version 2.53.0
pisto@fedora:~/Desktop/cripto/lab2$
```

Figura 3: demostración de la versión de GIT utilizada

2.1.2. Desarrollo de actividad

Se levantó una imagen de la aplicación Web DVWA, para esto fue necesario clonar el repositorio de DVWA desde la página **https://github.com/digininja/DVWA** usando git a través del siguiente comando:

```
pisto@fedora:~/Desktop/cripto/lab2$ git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5703, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 5703 (delta 25), reused 17 (delta 17), pack-reused 5671 (from 3)
Receiving objects: 100% (5703/5703), 2.72 MiB | 4.42 MiB/s, done.
Resolving deltas: 100% (2842/2842), done.
pisto@fedora:~/Desktop/cripto/lab2$
```

Figura 4: Clonación de repositorio DVWA

una vez clonado el repositorio se modificará el archivo **compose.yaml** para poder levantar correctamente el contenedor

```yaml
services:
  dvwa:
    build: .
    container_name: dvwa
    ports:
      - "8080:80"
    depends_on:
      - db
    environment:
      - DB_SERVER=db
      - DB_DATABASE=dvwa
      - DB_USER=dvwa
      - DB_PASSWORD=dvwa
    restart: unless-stopped

  db:
    image: mariadb:10.5
    container_name: dvwa_db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
    volumes:
      - dvwa_db_data:/var/lib/mysql

volumes:
  dvwa_db_data:
```

Figura 5: Docker Compose editado para el laboratorio

usando este composer se levantará una imagen de la pagina web en docker, en conjunto a una imagen de Mariadb para poder almacenar los datos de dicha base de datos, luego se levanta la imagen de los docker

```
pisto@fedora:~/Desktop/cripto/lab2/DVWA$ sudo docker-compose up --build -d
[+] up 11/11
 ✓ Image mariadb:10.5 Pulled
[+] Building 4.2s (16/16) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 510B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.11kB
 => [internal] load metadata for docker.io/library/composer:latest
 => [internal] load metadata for docker.io/library/php:8-apache
 => [internal] load .dockerignore
 => => transferring context: 179B
 => [internal] load build context
 => => transferring context: 34.61kB
 => [stage-0 1/7] FROM docker.io/library/php:8-apache@sha256:103a8cd59a029b387470deace9736a7bc5aa
 => => resolve docker.io/library/php:8-apache@sha256:103a8cd59a029b387470deace9736a7bc5aeb167a745
 => FROM docker.io/library/composer:latest@sha256:698d3801b2a622ace460c4743c781282fcbcb733a4cbf8b
 => => resolve docker.io/library/composer:latest@sha256:698d3801b2a622ace460c4743c781282fcbcb733a
 => CACHED [stage-0 2/7] WORKDIR /var/www/html
 => CACHED [stage-0 3/7] RUN apt-get update  && export DEBIAN_FRONTEND=noninteractive  && apt-get
 => CACHED [stage-0 4/7] COPY --from=composer:latest /usr/bin/composer /usr/local/bin/composer
 => CACHED [stage-0 5/7] COPY --chown=www-data:www-data . .
 => CACHED [stage-0 6/7] COPY --chown=www-data:www-data config/config.inc.php.dist config/config.
 => CACHED [stage-0 7/7] RUN cd /var/www/html/vulnerabilities/api  && composer install
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:48c8755b28c5ec3ea7ede778a63d9c48ea8a9b74e098765408629ee3746b2501
 => => exporting config sha256:8167b01d37dcd47927914605ed03c4278baeac69b35b86d8462cf76af4619f07
 => => exporting attestation manifest sha256:48e51623547555a4057afacb75a564b1eee9fbd041c0a4e053ee
 => => exporting manifest list sha256:ec51eebe8bb89c8db0ec4a3737d161dffb97ef8d88b7f7ddd90695bc234
 => => naming to docker.io/library/dvwa-dvwa:latest
[+] up 14/14king to docker.io/library/dvwa-dvwa:latest
 ✓ Image mariadb:10.5 Pulled
 ✓ Image dvwa-dvwa    Built
 ✓ Container dvwa_db  Started
 ✓ Container dvwa     Started
pisto@fedora:~/Desktop/cripto/lab2/DVWA$
```

Figura 6: Levantamiento de imagenes de docker para levantar la pagina DVWA

Se comprueba que se hayan armado correctamente las imágenes de la pagina web y la base de datos, y que los contenedores correspondientes estén corriendo.



Figura 7: Verificacioón de los contenedores correspondientes corriendo

finalmente accederemos a través del navegador a la dirección http://localhost:8080 para comprobar que la pagina esta arriba y funcionando.

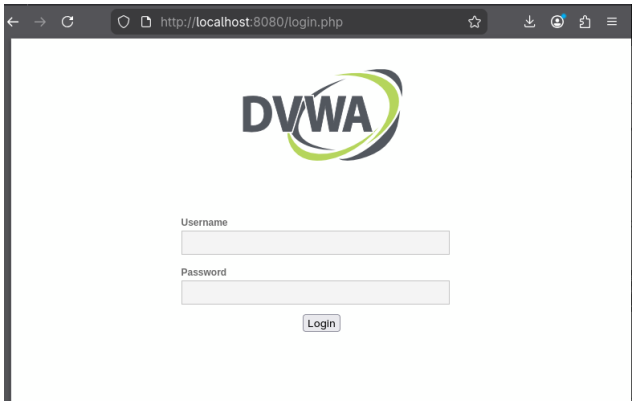


Figura 8: Landing page de DVWA levantada en localhost en el puerto 8080

2.2. Redirección de puertos en docker (dvwa)

la redireccción de los puertos de docker fue realizada durante en la sección anterior, dentro del docker compose usando la opción **ports: 8080:80**, abrirá los puertos 8080 y 80 para abrir los puertos **HTTP**, en este caso no será necesario usar el puerto **HTTPS** ya que el objetivo de este laboratorio no es analizar su cifrado.

2.3. Obtención de consulta a replicar (burp)

2.3.1. Dependencias

- **Burpsuite Community edition** para esta seccion fue necesario instalar la suite de pentesting Burpsuite, en este caso se usó la edición Community

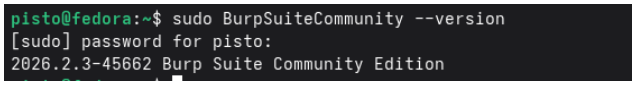


Figura 9: Version de Burpsuite Community Edition utilizada en el laboratorio

2.3.2. Desarrollo de actividad

una vez iniciado burpsuite se conectamos el proxy a la dirección 127.0.0.1:8081 para poder obtener el request a replicar a través del navegador que nos otorga burpsuite se accede a la pagina que se levantó en la actividad anterior, en específico a la dirección **http://localhost:8080/vulnerabilities/brute/** Se prendió el modo intercept de burpsuite y se intentó acceder con credenciales que no existen, con el objetivo de capurar el request que se envia por el método GET HTTP

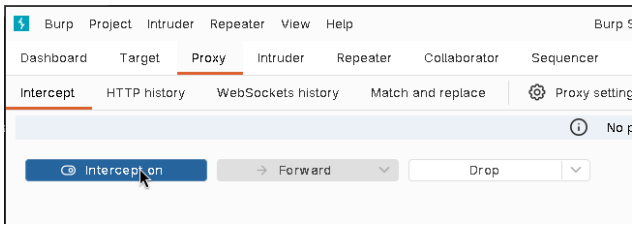


Figura 10: inicio del modo intercept en Burpsuite

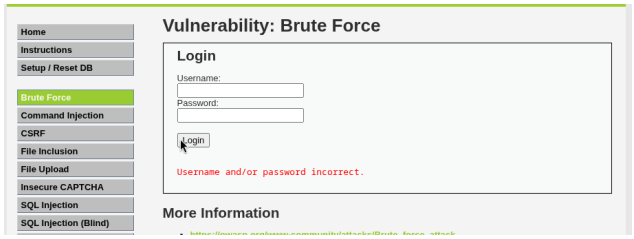


Figura 11: ingreso de credenciales incorrectas para capturar el request

aquí se puede observar el paquete de request capturado por burpsuite y que se utilizará mas adelante para armar el ataque de fuerza bruta.

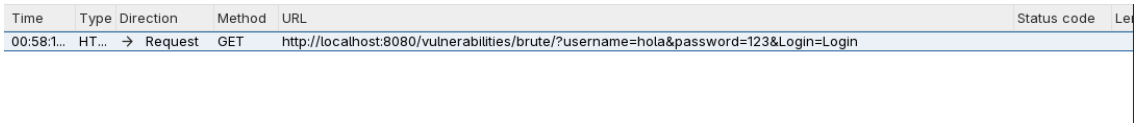


Figura 12: paquete capturado con Burpsuite

2.4. Identificación de campos a modificar (burp)

Para ejecutar el ataque de fuerza bruta, una vez capturada la consulta hacemos abriendo el menú del click derecho, se encuentra la opcion de enviar el paquete a la pestaña de **Intruder** lLuego de enviar el paquete entramos a la pestaña de Intruder podemos revisar el request, dentro de este se encuentran los campos enviados al tratar de verificar las credenciales.

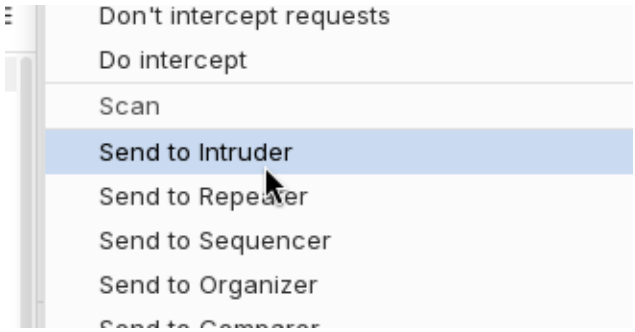


Figura 13: Envío del paquete a la pestaña Intruder

Se destacan los campos correspondientes a las credenciales enviadas al capturar el paquete y se selecciona la opción **Add**

2.5. Obtención de diccionarios para el ataque (burp)

Para que el ataque tenga un caracter profesional, seleccionaremos como diccionario para el ataque el diccionario publico **SecLists** que se encuentra en la url https://github.com/danielmiessler/seclis

```
pisto@fedora:~/Desktop/cripto/lab2$ git clone https://github.com/danielmiessler/SecLists.git
Cloning into 'SecLists'...
remote: Enumerating objects: 58931, done.
remote: Total 58931 (delta 0), reused 0 (delta 0), pack-reused 58931 (from 1)
Receiving objects: 100% (58931/58931), 3.04 GiB | 15.09 MiB/s, done.
Resolving deltas: 100% (42681/42681), done.
Updating files: 100% (6258/6258), done.
pisto@fedora:~/Desktop/cripto/lab2$
```

Figura 14: clonación de repositorio Seclists de GIT

los diccionarios se eligios contienen una lista de las contraseñas más comunes que no tiene un carácter altamente extenso, por lo tanto el cómputo de las requests no se extenderá mucho. En esta ocasión se utilizarán los archivos **cirt-default-usernames.txt** para los usuarios; y el archivo **10k-most-common.txt** para las contraseñas

2.6. Obtención de al menos 2 pares (burp)

Los archivos de la sección anterior se usaron en Burpsuite para hacer el ataque por fuerza bruta, seleccionando en la pestaña de Payloads la listas de usuarios y contraseñas que elegimos en la sección anterior

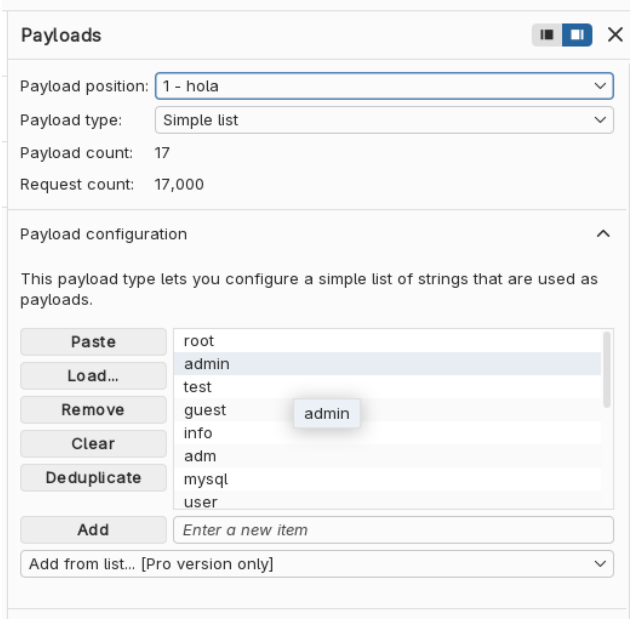


Figura 15: Sección de listado de usuarios usados para el ataque por fuerza bruta

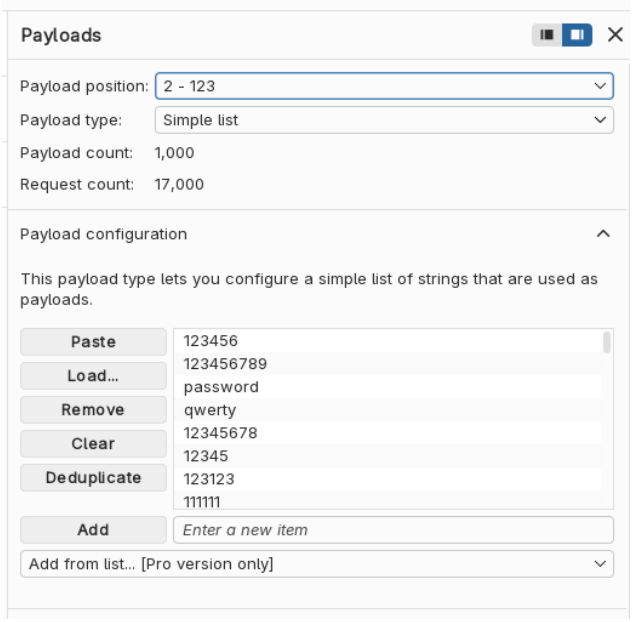


Figura 16: Sección de listado de contraseñas usadas para el ataque por fuerza bruta

luego de seleccionar los archivos iniciamos el ataque.

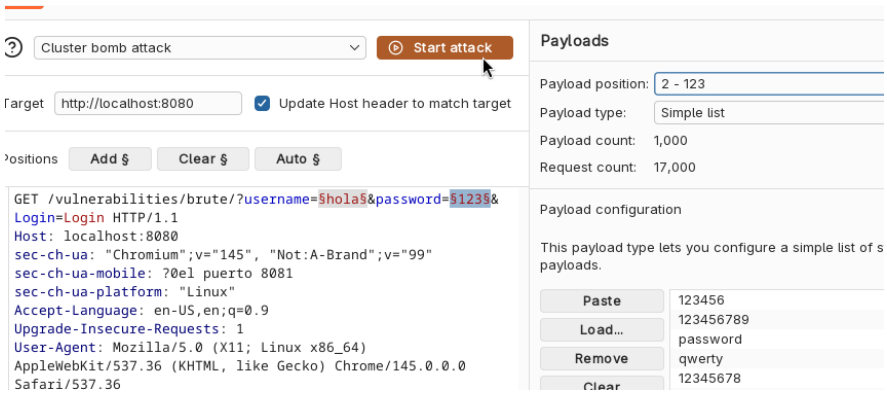


Figura 17: Enter Caption

durante el ataque burpsuite empareja cada username con cada contraseña desde los archivos, preparando un request para cada uno y enviandolos de manera secuencial, de esta manera generando el ataque por fuerza bruta.

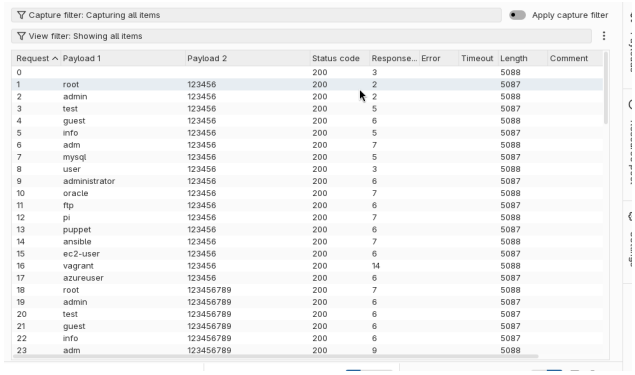


Figura 18: Ataque de fuerza bruta siendo ejecutado por BurpSuite

Tras dejar corriendo el ataque por fuerza bruta durante 30 minutos, se observó que existen requests con respuestas de un largo diferente al resto, estas requests corresponden a requests existosas, demostrando que el ataque de fuerza bruto tuvo resultados exitosos

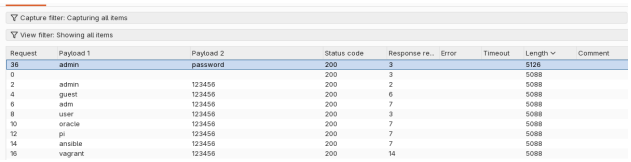


Figura 19: ejemplo de request con diferente largo

estas son dos capturas que resultaron ser exitosas

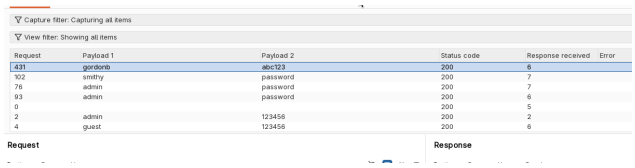


Figura 20: Ejemplo 1 de request exitosa por ataque

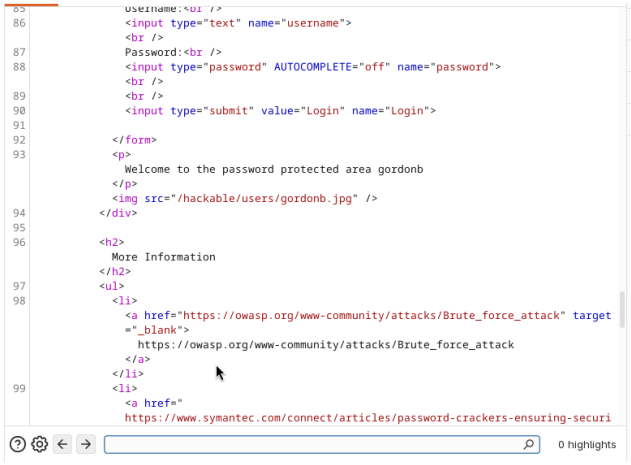


Figura 21: Evidencia de respuesta exitosa de Ejemplo 1

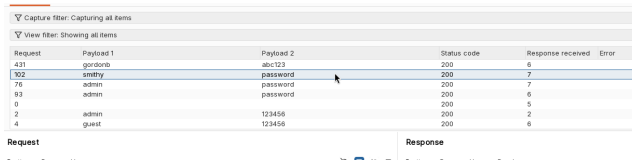


Figura 22: Ejemplo 2 de request exitosa por ataque

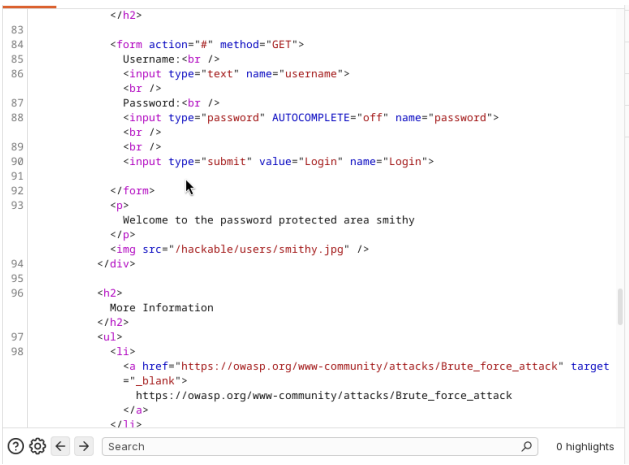


Figura 23: Evidencia de respuesta exitosa de Ejemplo 2

2.7. Obtención de código de inspect element (curl)

para la siguiente sección del laboratorio accederemos nuevamente a la pagina de fuerza bruta de DVWA, se hará un intento de login con el objetivo de capturar el request a inspeccionando elementos en el navegador

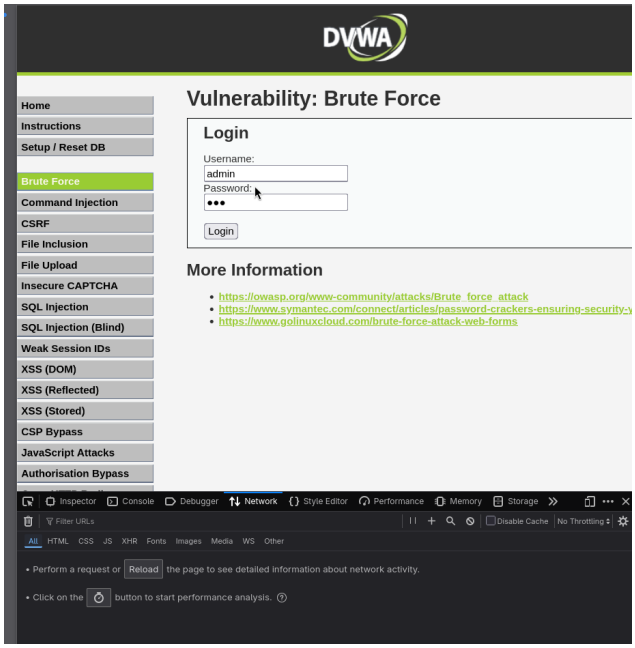


Figura 24: inspeccion de elemento en navegador

en la pestaña de network podremos copiar el codigo necesario para continuar con la actividad

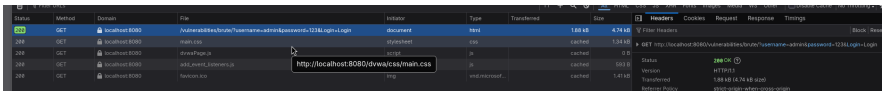


Figura 25: Request capturada en inspección

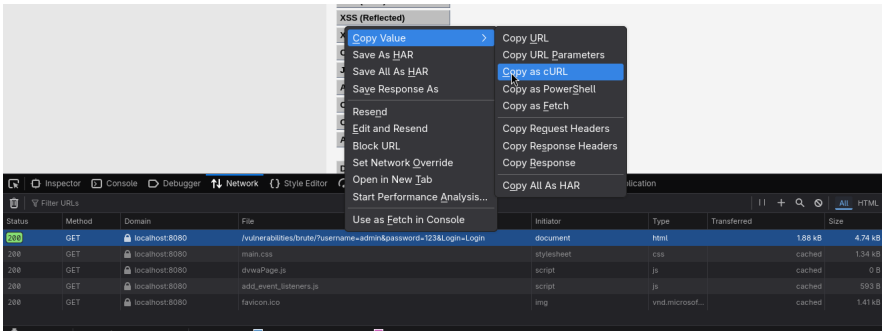


Figura 26: copia de request con cURL

2.8. Utilización de curl por terminal (curl)

En la siguiente imágen se puede ver la cURL copiada en la seccion anterior, esta será ejecuada con el parametro -v para ver el modo verbose



Figura 27: comando Curl usado por terminal con credenciales válidas



Figura 28: verbose de la respuesta con credenciales válidas

```
        <form action="#" method="GET">
                Username:<br />
                <input type="text" name="username"><br />
                Password:<br />
                <input type="password" AUTOCOMPLETE="off" name="password"><br />
                <br />
                <input type="submit" value="Login" name="Login">

        </form>
        <p>Welcome to the password protected area admin</p><img src="/hackable/users/admin.jpg" />
</div>

<h2>More Information</h2>
<ul>
        <li><a href="https://owasp.org/www-community/attacks/Brute_force_attack" target="_blank">https://owa
        <li><a href="https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-pass
r-password</a></li>
```

Figura 29: comando Curl usado por terminal con credenciales inválidas

```
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* using HTTP/1.x
> GET /vulnerabilities/brute/?username=admin&password=123&Login=Login HTTP/1.1
> Host: localhost:8080
> Cookie: PHPSESSID=592b030997a446171d4d5608d588342e; security=low
> Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
> Accept-Language: en-US,en;q=0.9
> Cache-Control: max-age=0
> Proxy-Connection: keep-alive
> Referer: http://localhost:8080/vulnerabilities/brute/
> Sec-Fetch-Dest: document
> Sec-Fetch-Mode: navigate
> Sec-Fetch-Site: same-origin
> Sec-Fetch-User: ?1
> Upgrade-Insecure-Requests: 1
> User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
> sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
> sec-ch-ua-mobile: ?0
> sec-ch-ua-platform: "Linux"
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 14 Apr 2026 04:56:13 GMT
< Server: Apache/2.4.66 (Debian)
< X-Powered-By: PHP/8.5.5
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
< Vary: Accept-Encoding
< Content-Length: 4735
< Content-Type: text/html;charset=utf-8
```

Figura 30: verbose de la respuesta con credenciales inválidas

2.9. Demuestra 4 diferencias (curl)

dentro de las diferencias que podemos encontrar entre una payload exitosa y una payload no exitosa estan:

- **Largo de la payload:** como se puede ver en las imágenes anteriores la payload de respuesta tienen un lago diferente cuando la respuesta a esta cambia



Figura 31: largo de respuesta a payload válida

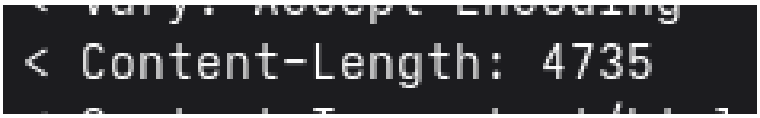


Figura 32: largo de respuesta a payload inválida

- **mensaje de login exitoso:** el login exitoso muestra como respuesta un mensaje diferente a cuando las credeciales no son correctas

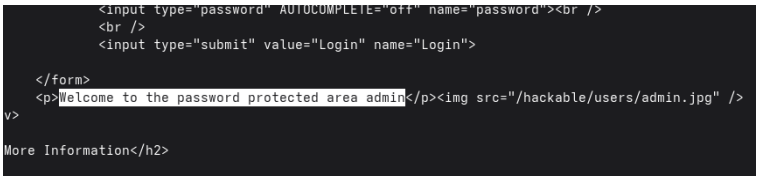


Figura 33: Mensaje emitido al lograr validar credenciales exitosamente



Figura 34: mensaje de credenciales rechazadas

- **imagen:** al hacer un login exitoso se muestra una imagen, este campo no está presente cuando el login no es exitoso



Figura 35: Imagen agregada al login exitoso

- **Etiquetas php:** las etiquetas php del mensaje que se muestra al tratar de hacer login son diferentes, el login exitoso usa la etiqueta del tipo **¡p¿** mientras que un logn fallida usa etiquetas **¡pre¿** esto se puede apreciar en las figuras 33 y 34

2.10. Instalación y versión a utilizar (hydra)

actualmente el ambiente en el que se está trabajando es en la distribucion de Fedora de linux, por lo tanto para instalar Hydra usaremos la libreria de manejo de packages DNF



Figura 36: comando para instalar Hydra

La version de hydra que se utilizó en este labortorio es la 9.5 como se puede ver a continuación



Figura 37: versión de Hydra utilizada en el laboratorio

2.11. Explicación de comando a utilizar (hydra)



Figura 38: comando Hydra para obtener contraseñas

Se utilizó el módulo http-get-form de Hydra para atacar el formulario . Se definieron listas de usuarios y contraseñas, junto con los parámetros de la petición HTTP. Se incluyó

una cookie de sesión válida para mantener autenticación en DVWA, y se configuró un mensaje de error para que Hydra identifique intentos fallidos.

2.12. Obtención de al menos 2 pares (hydra)

lamentablemente hydra no fué util en esta experiencia para poder obtener pares de contraseñas dado que al tratar de usarlo para obtener pares, este consideraba todos los pares de usuario contraseña como pares válidos

```
[8080][http-get-form] host: localhost   login: root   password: monkey
[8080][http-get-form] host: localhost   login: root   password: password
[8080][http-get-form] host: localhost   login: root   password: 1234
[8080][http-get-form] host: localhost   login: root   password: qwerty
[8080][http-get-form] host: localhost   login: root   password: 12345678
[8080][http-get-form] host: localhost   login: root   password: dragon
[8080][http-get-form] host: localhost   login: root   password: 123456
[8080][http-get-form] host: localhost   login: root   password: letmein
[8080][http-get-form] host: localhost   login: root   password: abc123
[8080][http-get-form] host: localhost   login: root   password: mustang
[8080][http-get-form] host: localhost   login: root   password: football
[8080][http-get-form] host: localhost   login: root   password: 696969
[8080][http-get-form] host: localhost   login: root   password: 12345
[8080][http-get-form] host: localhost   login: root   password: baseball
[8080][http-get-form] host: localhost   login: root   password: michael
[8080][http-get-form] host: localhost   login: root   password: pussy
[8080][http-get-form] host: localhost   login: admin   password: password
[8080][http-get-form] host: localhost   login: admin   password: 123456
[8080][http-get-form] host: localhost   login: admin   password: qwerty
[8080][http-get-form] host: localhost   login: admin   password: 12345678
[8080][http-get-form] host: localhost   login: admin   password: 1234
[8080][http-get-form] host: localhost   login: admin   password: 12345
```

Figura 39: Falsos positivos entregados por Hydra

se sospecha queel comando no esta tomando en cuenta correctamente el paramero F, pero cualquier intento de edición de este comando resulta en errores similares

2.13. Explicación paquete curl (tráfico)

utilizando wireshark filtramos las comunicaciones que usan el puerto 8080 y al usar el comando curl podemos ver los paquetes generados

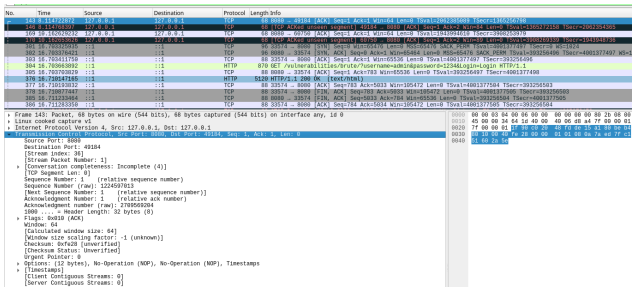


Figura 40: Paquetes generados por cURL capturados en wireshark

estos paquetes tienen un header natural, ya que fueron copiados directamente desde una request real, lo que podría llegar a delatar un ataque con esta herramienta es la repetición de

los parámetros del payload ya que vienen directamente de una consulta real. en términdos de frecuencia, no es diferenciable, ya que cUrl no es una herramienta automatizada

2.14. Explicación paquete burp (tráfico)

de la misma forma que en la sección anterior podemos ver

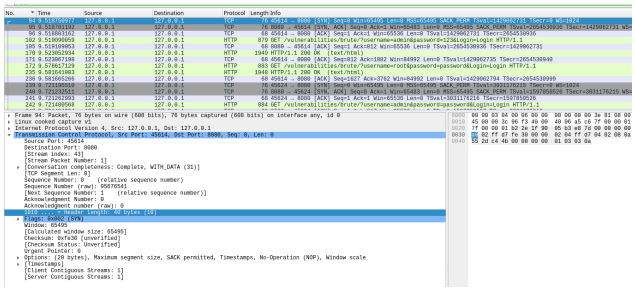


Figura 41: Paquetes caturados enviados por Burp

Se puede notar en el Wireshark, que Burp tiene solicitudes cons cabeceras más complejas que hydra y patrones de envío ajustables.

2.15. Explicación paquete hydra (tráfico)

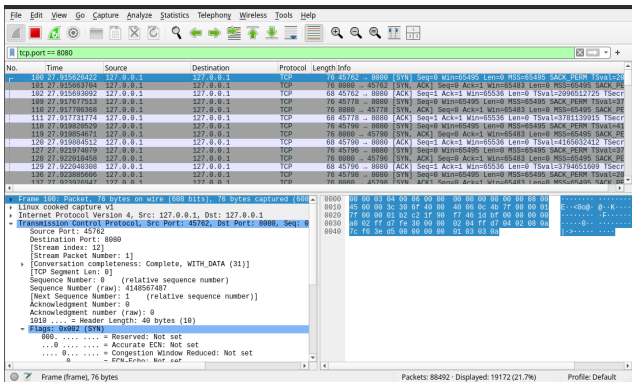


Figura 42: paquetes capturados enviados por Hydra

2.16. Mención de las diferencias (tráfico)

Se comparó el tráfico generado por Hydra, Burp Suite y cURL utilizando Wireshark. Se observó que Hydra genera muchas solicitudes en corto tiempo, lo que hace evidente su carácte de ataque de fuerza bruta automatizado. Burp Suite también realiza múltiples solicitudes, pero a menor velocidad y con mayor control sobre los parámetros. Por otro lado, cURL genera tráfico aislado y manua. Por otro lado, el campo User-Agent permitió distinguir claramente

cada herramienta, mientras que la frecuencia y repetitividad de las solicitudes fueron los principales indicadores de comportamiento sospechoso

2.17. Detección de SW (tráfico)

Se puede observar al inspeccionar los paquetes recibidos ya que se pueden observar los métodos en las cabeceras como User-Agent, en el cURL, suele mostrar pocas cabeceras, en BurpSuite, hereda cabeceras más "naturales"de navegadores como Chrome/Firefox, pero Hydra, usa cabeceras minimas y repetitivas. Ritmo de envio, cURL es secuencial y estable y depende del usuario su frecuencia, Burp puede configurarse para tener comportamientos y tiempos más "humanos", en el caso de hydra, va en rafagas concurrentes. En temas de conexiones TCP, Hydra por ejemplo abre muchas conexiones, y va rotando puertos, cURL y BurpSuite por como trabajan tienen a reutilizar puertas que en algún momento se utilizaron.

2.18. Interacción con el formulario (python)

2.19. Cabeceras HTTP (python)

2.20. Obtención de al menos 2 pares (python)

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

Se comparó el rendimiento de cURL, Hydra y Burp Suite en ataques de fuerza bruta. Se observó que cURL presenta un rendimiento bajo al ejecutar solicitudes de forma manual, mientras que Burp Suite permite automatización pero con limitaciones en su versión Community. Por otro lado, Hydra posee un gran potencial ya que es bastante mas rápido que las otras dos opciones usadas en este laboratorio.

2.22. Demuestra 4 métodos de mitigación (investigación)

- **MFA (autenticación multifactor).** Básicamente la autenticación multifactor es que dos o más factores de verificación para acceder a una cuenta. Funciona aún así si un atacante consigue adivinar la contraseña, que sería ese el primer factor, luego pasa por el segundo factor que podría ser es algo que el usuarioposee, como por ejemplo un código temporal enviado por SMS. Con respecto a los escenarios más eficaces, la defensa es lo más robusta contra los ataques de fuerza bruta, incluyendo, las que ocupan contraseñas débiles.

- **HASHING CON SAL** Este método básicamente, busca que las contraseñas sean lo más difíciles posibles de adivinar aún así si el atacante consigue acceder a la base de datos con las contraseñas hasheadas. Con respecto a los escenarios eficaces, lo ideal es proteger las credenciales en la base de datos y evitar que se obtengan masivamente las contraseñas, como lo puede ser con una tabla Rainbow.

- **CAPTCHA** Los CAPTCHA son pruebas diseñadas para distinguir a los usuarios normales (personas) de bots automatizados. Se le pide generalmente al usuario que complete una tarea sencilla que resulta difícil hacer a un software automatizado, como escribir texto distorsionado o identificar imágenes. Escenario eficaz, sirve mucho para el ámbito en el cual se necesita bloquear bots o herramientas automatizadas como lo vimos en este laboratorio.

- **Rate Limiting y Bloqueo de cuenta** Este método se centra en el número de intentos fallidos al iniciar o tratar de iniciar una sesión en un periodo de tiempo limitado generalmente. Muchas veces se le agrega un Rate Limiting, que es introducir un retardo temporal que incrementa dependiendo los intentos fallidos que se tengan e un intervalo de tiempo. Escenario eficaz, este es muy bueno contra ataques de fuerza bruta o automatizados ya que hace muy lento el proceso para poder "descubrir"la contraseña o las credenciales que se quieran buscar.

Conclusiones y comentarios

Como conclusión pudimos realizar un aaque de fuerza bruta en 3 tipos diferentes de herramientas y cada una tiene su nicho, cUrl sirve para ataques más controlados y precisos ya que su manera de trabajo es manual, pero es bastante personalizable, Burpsuite es mucho más intuitivo que el resto de herramientas, sin embargo sigue sin ser la opción más rápida para este tipo de ataques, Hydra es por lejos la opción mas rápida para este tipo de ataques ya que ofrece la posibilidad de paralelizar el computo de el ataque de fuerza bruta, sin embargo esto puede saturar los puertos y ser muy evidente si lo que se requiere es sigilo también se analizaron métodos de mitigación para evitar este tipo de ataques, y si bien hay métodos más eficaces que otros , nada será mas eficaz que colocar varias capas defensa en una aplicacion o equipo para asegurar la integridad de los datos de esta.